BULKING APP • AI MIGRATION PACKAGE

Technical Architecture & Database Specification

Document 03 of 07 • Version 1.0 • Audit snapshot: 21 October 2026

Purpose
Describe the verified GitHub structure, major application components, Neon database tables, relationships, configuration conventions and safe change procedures so a receiving AI can understand the current implementation before editing it.

Audit basis: GitHub default branch and files were inspected, and the connected Neon project's branch and public table schemas were queried read-only. This is a point-in-time snapshot; re-check before making changes.

Source	Observed value
GitHub repository	rd2080/Bulk_App_Personalised — public repository; default branch main
Observed main HEAD	2a1f25f31eb986fe44409c2adb860a48fd2d7921 — “Document deterministic meal command architecture”
Neon project	Bulk_App_Personalised; project ID ancient-bonus-73336167
Neon branch	production — branch ID br-old-waterfall-b3cas5au; primary/default and ready at audit time
Database platform	Neon PostgreSQL; project reported PostgreSQL 18
Schema observed	Eight public base tables; schemas inspected for all eight

GitHub main and Neon production are separate systems. A SQL migration committed to GitHub does not apply itself to Neon, and the live Streamlit deployment version must be checked independently.

1. Architecture overview

The app is a single-user Streamlit application written in Python. It uses Neon PostgreSQL for persistence and a repository pattern for database operations. The source repository contains UI pages, service/domain helpers, AI provider adapters, SQL migrations, tests and documentation.

1.1 High-level data flow

UI: browser → Streamlit pages (`app.py` and `pages/*.py`) → validation/services → repository modules → `bulking_app/neon.py` → Neon PostgreSQL.

Voice: browser speech recognition or typed text → user reviews/corrects transcript → deterministic parser → validation/clarification → meal-plan repository → `daily_meal_plans` and `planned_meals`.

AI: request builder and provider adapters are present. The repository documentation says AI meal suggestions and nutrition calculations remain future work. Presence of adapter files is not proof that end-user AI planning is implemented.

1.2 Established architecture principles

- GitHub stores source code, SQL migrations, tests and documentation.
- Streamlit provides separate user workflows rather than one monolithic screen.
- bulking_app/neon.py is the shared Neon connection helper.
- Database operations are organised in repository modules.
- Deterministic meal commands use Python rules and do not require a Gemini/OpenAI call.
- AI request preparation is separate from provider execution.
- Schema changes are versioned in numbered SQL migration files.

1.3 Functional areas described by the current README

- Food Inventory and recipe library, including CSV import and downloadable template.
- Morning check-in and weight history.
- Single-user profile management.
- Browser-based voice transcription with editable text and typed fallback.
- Daily meal-plan create, edit, remove and review workflows.
- Deterministic commands for adding, replacing and removing planned meals.

The README identifies AI meal suggestions, nutrition calculations and actual-consumption tracking as future work. This is a key distinction for the migration.

2. Repository map

The following paths were observed in the GitHub main tree. This is a concise architectural map, not a complete file listing.

Path	Responsibility
app.py	Streamlit application entry point.
pages/1_Food_Inventory.py	Food inventory and recipe library workflow.
pages/1_Profile.py	Single-user profile editor.
pages/2_Morning_Check-in.py	Daily check-in interface.
pages/3_Voice_Input.py	Browser transcription, text review and deterministic command workflow.
pages/4_Daily_Meal_Plan.py	Dated meal-plan interface.
bulking_app/neon.py	Shared Neon connection helper.
bulking_app/database.py	Database-related helper; inspect current use before changing.
bulking_app/meal_commands.py	Deterministic parsing and validation of supported commands.
bulking_app/voice_input.py	Voice input support.
bulking_app/repositories/	Repositories for inventory, meal plans, check-ins, recipes and profile.
bulking_app/services/	Recipe CSV service and other application helpers.

Path	Responsibility
bulking_app/ai/	Base adapter, factory, Gemini adapter, OpenAI adapter and request builder.
bulking_app/components/browser_speech/	Browser speech component.
database/migrations/	Numbered PostgreSQL migrations 001–008 and migration guidance.
docs/	Architecture, schema, phase, voice, recipe and decision documentation.
tests/	Tests for inventory, commands, plans, check-ins, recipes, request builder, services and profile.
requirements.txt / .env.example	Dependencies and configuration variable names; example secrets are blank.

3. Runtime and configuration

3.1 Local startup described by README

1. Use Python 3.9 or later, as documented in README.
2. Install dependencies from requirements.txt.
3. Configure DATABASE_URL with the Neon PostgreSQL connection string in local .env or the deployment secret store.
4. Run `streamlit run app.py`.

requirements.txt currently declares Streamlit, python-dotenv, psycopg[binary], google-genai and openai with bounded major versions. Check the installed environment before changing dependencies.

3.2 Configuration variables in .env.example

Variable	Purpose / note
DATABASE_URL	Neon PostgreSQL connection string. Secret; never commit a populated value.
GEMINI_API_KEY	Gemini credential; blank in example.
OPENAI_API_KEY	OpenAI credential; blank in example.
AI_PROVIDER	Provider selection variable; example value is gemini.
GEMINI_MODEL	Example value gemini-2.5-flash; verify current provider support before changing.
OPENAI_MODEL	Example value gpt-4.1-mini; verify current provider support before changing.

Never include real secret values in handover PDFs, logs, GitHub issues, committed prompts or screenshots. The receiving AI needs variable names and setup instructions, not copied credentials.

3.3 Integration boundaries

- Neon is accessed through the shared connection helper; repository modules should own query logic.
- Voice transcription occurs in the browser component and the transcript is reviewed before commands are applied.
- Deterministic commands do not require an AI-provider call.
- The request builder prepares profile-aware context; adapters encapsulate provider-specific behaviour.
- Verify actual call sites before assuming AI_PROVIDER controls every workflow.

4. Live Neon schema inventory

The connected Neon project listed these eight public base tables at audit time. Column definitions and constraints were inspected. Re-check the live schema before a migration.

Table	Purpose	Relationships / notes
food_inventory	Food items and availability/stock metadata.	Referenced by recipe_ingredients; deletion restricted while used by a recipe.
weight_entries	Weight history.	Unique entry_date; current documented range 25–350 kg.
daily_checkins	One check-in per date.	Unique checkin_date; weight, sleep, soreness and notes; syncs weight to weight_entries.
user_profile	Single current profile.	Ten fields; unique expression index on constant true limits it to one row.
recipe_library	Recipe metadata and recipe-level nutrition.	Unique recipe_name; ingredients stored separately.
recipe_ingredients	Recipe-to-food composition.	Composite key (recipe_id, food_id); recipe cascade and food delete restriction.
daily_meal_plans	One dated plan header.	Unique plan_date; parent of planned_meals.
planned_meals	Ordered meals within a plan.	plan_id references daily_meal_plans; child rows cascade on plan deletion.

No dedicated actual-consumption meal-log table appeared in the live table list. Do not describe planned_meals as actual consumption. Workout logs, reminder schedules and nutrition target history were also not present in this observed table inventory.

5. Table-level data dictionary

5.1 weight_entries

Columns: id BIGINT/BIGSERIAL primary key; entry_date DATE NOT NULL UNIQUE; weight_kg NUMERIC(5,2) NOT NULL; created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(). The current documented schema constrains weight to 25–350 kg. Migration 001 used too-narrow NUMERIC(3,2); migration 005 widens precision and adds the range constraint. Do not restore the old type.

5.2 daily_checkins

Columns: id INTEGER identity primary key; checkin_date DATE NOT NULL UNIQUE; weight_kg NUMERIC(5,2) nullable for legacy compatibility; sleep_hours NUMERIC(3,1) nullable; soreness_level INTEGER nullable; daily_notes TEXT nullable; created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(). Checks constrain weight to 25–350 kg when present, sleep to 0–24 hours and soreness to 1–10. New check-ins require weight. The repository upserts the corresponding date and weight into weight_entries in the same transaction.

5.3 user_profile

Exactly ten columns: age SMALLINT; height_cm NUMERIC(5,1); target_weight_kg NUMERIC(5,1); diet_type TEXT; diet_philosophy TEXT; calorie_strategy TEXT; cuisine_preference TEXT; regional_context TEXT; cooking_complexity TEXT; meals_per_day SMALLINT. Fields are NOT NULL and validated by constraints. A unique expression index on (true) enforces at most one row. There is no id, name, timestamp, current weight, primary_goal, allergies or foreign-food-preference column.

5.4 food_inventory

Columns: id INTEGER primary key; food_name TEXT NOT NULL; availability_type TEXT NOT NULL; quantity NUMERIC nullable; unit TEXT nullable; is_available BOOLEAN NOT NULL DEFAULT TRUE; created_at and updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(). availability_type is constrained to always_available, stock_tracked or on_demand. Inspect the full live quantity constraint before changing validation. This is the canonical food list used by recipe composition.

5.5 recipe_library and recipe_ingredients

recipe_library columns: id, recipe_name, recipe_type, preference_level, cooking_complexity, total_weight_g, calories_kcal, protein_g, carbohydrates_g and fat_g. Recipe names are unique. Supported types are meal/snack/shake; preference values favorite/like/neutral/dislike/never; complexity values easy/medium/hard. Prepared weight must be positive and nutrient values non-negative.

recipe_ingredients columns: recipe_id, food_id, quantity and unit. Primary key (recipe_id, food_id); quantity positive; unit non-empty. recipe_id references recipe_library with ON DELETE CASCADE; food_id references food_inventory with ON DELETE RESTRICT. Ingredient rows do not store calories/macros. Serving nutrition scales recipe totals proportionally using serving_weight_g / total_weight_g.

5.6 daily_meal_plans and planned_meals

daily_meal_plans stores id, plan_date, created_at and updated_at, with unique plan_date. planned_meals stores id, plan_id, meal_type, meal_name, description, position and created_at. Meal types are Breakfast, Morning snack, Lunch, Afternoon snack, Dinner or Other. Meal name length is limited to 1–160 trimmed characters and description to 2,000 characters. Each meal references a plan with ON DELETE CASCADE and has an index supporting ordered retrieval.

6. Entity relationships

Weight: daily_checkins stores one row per check-in date and the repository upserts the same date/weight into weight_entries.
Recipes: food_inventory 1-to-many recipe_ingredients; recipe_library 1-to-many recipe_ingredients. Recipe deletion cascades to ingredient rows; food deletion is restricted while referenced.
Plans: daily_meal_plans 1-to-many planned_meals, ordered by position and id.

There is no dedicated actual-consumption meal-log relationship in the inspected live schema. A planned meal must not be counted as eaten. This distinction must be preserved when implementing future meal tracking.

7. Migration policy and schema evolution

- Numbered SQL files in database/migrations/ are the versioned source of truth for intended schema changes.
- The repository contains migrations 001–008 for weight entries, check-ins, inventory, weight-on-check-in, weight precision correction, profile, recipe library and dated meal plans.
- Applied migration files should be treated as immutable; create a new migration for new changes.
- Update schema documentation and relevant tests in the same change.
- Apply migrations to the intended Neon branch/database before deploying code that depends on them.
- Committing a migration to GitHub does not change the live Neon database.
- Prefer additive changes and preserve user data. Destructive changes require approval, recovery planning and dependency analysis.
- Verify live constraints and foreign keys rather than relying only on migration files.

7.1 Migration checklist

1. Confirm the intended project, branch and database.
2. Inspect the live schema and the migration's effect; trace code references and foreign keys.
3. Assess rerun safety, locks and data rewrites.
4. Protect important data before risky changes.
5. Apply the migration through the approved process.
6. Re-inspect schema and run targeted tests.
7. Update documentation and record migration/test evidence.

8. Security and reliability rules

- Never commit DATABASE_URL or provider keys. Use local environment configuration or deployment secrets.
- Use parameterised SQL and validate inputs, including CSV rows and AI output.
- Use transactions when related writes must succeed or fail together.
- Use read-only access for audits where practical.
- Do not drop tables, reset branches, truncate records or run destructive SQL without owner approval.
- Keep errors actionable without exposing credentials or sensitive connection details.
- Handle provider errors separately from database errors; do not silently lose an action when AI fails.

9. Known gaps against the broader vision

- Actual-consumption meal logging is not represented by a dedicated table in the observed schema.
- AI meal suggestions and nutrition calculations are documented as future phases, even though provider adapter files exist.
- Workout logging, reminders and adaptive analytics are not evidenced by the current table inventory.
- Recipe nutrition is stored at recipe level; ingredient rows contain composition only.
- Deterministic meal commands update planned meals; they do not normalise ingredients or record consumption.
- Streamlit deployment state and the commit running live were not independently verified in this audit.

These are observed gaps against the intended product vision, not automatically defects. Compare them with the roadmap and current acceptance criteria.

10. What the receiving AI must verify before coding

1. Refresh GitHub main HEAD and inspect the working tree. Audit snapshot HEAD was 2a1f25f31eb986fe44409c2adb860a48fd2d7921.
2. Inspect full source for entry point, repository methods, provider adapters and command parsing before changing interfaces.
3. Confirm Neon project and that production branch br-old-waterfall-b3cas5au is still the intended target.
4. Inspect complete live constraints if touching inventory, recipe or planned_meals logic.
5. Compare migrations 001–008 with the live schema; file presence does not prove migration application.
6. Run tests in a clean environment and record actual results. Tests exist, but this documentation audit did not execute them.
7. Verify Streamlit deployment and deployed commit if deployment work is requested.

11. Copy-ready instruction for the incoming AI

This document records the observed architecture and database at audit time. Refresh GitHub HEAD and the Neon schema before editing. Compare live schema with migrations and documentation. Preserve the single-user Streamlit/Python architecture and Neon persistence unless an approved requirement calls for change. Keep database queries in repositories, provider-specific logic behind adapters and deterministic meal commands separate from model calls. Treat daily_meal_plans/planned_meals as plans only, not actual-consumption logs. Use additive migrations, protect existing data, update docs and tests together, and verify the live target before any migration. Never expose secrets or claim tests/deployment succeeded without evidence.

12. Audit record

Item	Evidence recorded
GitHub default branch	main
Observed main commit	2a1f25f31eb986fe44409c2adb860a48fd2d7921; “Document deterministic meal command architecture”
Neon project	Bulk_App_Personalised; project ID ancient-bonus-73336167
Neon branch	production; br-old-waterfall-b3cas5au; primary/default and ready at audit time
Live tables	Eight public tables; column-level schema inspected
Tests / live app	Not executed or independently verified during this documentation audit

End of Document 03 • Next: Functional Requirements & User Workflows